Exercice Domaine Design et Produits

Demarche globale

Ce repository contient mes reponses a l'exercice "Domaine Design et Produits".

Hypotheses de travail

L'exercice ne precise pas tout. Voici mes hypotheses :

- Le projet a demarre il y a ~3 mois, avec un horizon total de ~12 mois
- Le budget total est calibre pour l'equipe actuelle (9 devs) + infra + licences
- La plateforme est en production avec des utilisateurs actifs
- Les donnees patients sont pseudonymisees conformement au cadre reglementaire (RGPD, HDS)
- L'hebergement on-premise est sur un datacenter APHP certifie HDS
- Le product owner / responsable metier existe mais n'est pas mentionne
- Le DSN (Directeur des Services Numeriques) est mon sponsor direct

---

Quelles analyses menez-vous et comment organisez-vous les 3 premieres semaines ?

J'arrive sur un projet en difficulte : 3 mois ecoules, 25% du budget consomme, 15% des fonctionnalites livrees. Avant de changer quoi que ce soit, je dois comprendre.

Mon principe : les 3 premieres semaines, j'observe, j'ecoute, je diagnostique. Je n'agis que sur les urgences bloquantes.

Semaine 1 : Ecouter et comprendre

Entretiens individuels (1h chacun, tous les devs, en toute confidentialite). Je cherche a comprendre la charge reelle, les blocages quotidiens, ce qui marche, et le moral de chacun.

Entretiens parties prenantes :

- **DSN** : Attentes precises, contraintes politiques, marge de manoeuvre
- **Product Owner** : Priorites fonctionnelles, frustrations
- **2-3 utilisateurs cles** (medecins/chercheurs) : Usage reel, irritants
- **Ops / infra** : Etat du cluster K8S, incidents recents

Audit technique rapide (un scan, pas un audit complet) :

- Les 10 requetes SQL les plus lentes (`pg_stat_statements`)
- Pipeline CI/CD : historique des 30 derniers deploiements, taux d'echec
- Couverture de tests reelle (tests qui passent vs tests desactives)
- Les 120 tickets de dette : tri rapide en bloquant / structurant / cosmetique
- Coherence de l'architecture Django/FastAPI, points de fragilite

Livrable : document de diagnostic brut (carte des risques, top 5 problemes techniques, estimation de la velocite reelle).

Semaine 2 : Analyser et prioriser

Categorisation de la dette technique :

Categorie	Volume estime	Action
Bloquant	~15-20 tickets	Sprint en cours
Structurant	~30-40 tickets	Budget de 20% par sprint
Cosmetique	~60-70 tickets	Backlog, traite en opportunite

Plan de stabilisation en 3 axes :

1. **Performance** : les requetes > 6s, c'est le probleme numero 1 visible par les utilisateurs
2. **Deploiements** : stabiliser la CI/CD pour arreter les rollbacks
3. **Equipe** : adresser les tensions

Livrable : plan de bataille (90 jours) partage avec le DSN -- objectifs mesurables a 30/60/90 jours, arbitrages proposes, risques majeurs.

Semaine 3 : Cadrer et lancer

Cadre de delivery :

- Sprint de 2 semaines, decoupage 80% feature / 20% dette

- Definition of Done clarifiee : pas de merge sans tests, pas de deploy sans review

Rituels :

Rituel	Frequence	Duree	Objectif
Daily standup	Quotidien	15 min	Deblocage
Sprint planning	Bi-hebdo	2h	Engagement sprint
Sprint review	Bi-hebdo	1h	Demo, feedback
Retro	Bi-hebdo	1h	Amelioration continue
Comite delivery	Hebdo	30 min	Arbitrages, risques

Premier sprint : 1-2 quick wins performance, 1 action CI/CD (gate de tests avant merge), le reste en features prioritaires.

Communication :

- A l'equipe : "Voici ce que j'ai compris, ce qu'on va faire, et pourquoi"
- Au DSN : "Voici l'etat reel, mon plan, et ce dont j'ai besoin"
- Pas de promesse sur les delais tant que je n'ai pas vu un sprint complet tourner

---

Ce que je ne fais PAS ces 3 premieres semaines

- Je ne reorganise pas l'equipe
- Je ne change pas l'architecture
- Je ne promets pas de dates
- Je ne blame personne

L'objectif : comprendre la realite, construire la confiance, poser les bases d'un delivery maitrise.

Comment augmenter la couverture des tests et stabiliser les deploiements ?

Cibles

Indicateur	Actuel	Cible 3 mois	Cible 6 mois
Couverture de tests	45%	65%	75%
Taux de rollback	Frequent	< 10%	< 5%
Incidents post-deploy	Non mesure	< 2/mois	< 1/mois

Je ne vise pas 90%. 75% sur le code critique, c'est plus utile que 90% de tests superficiels.

Augmenter la couverture de tests

Jamais de "sprint de tests". Ca ne marche pas et ca demotive. La couverture se construit de 3 facons :

1. Regle du boy-scout : tout code touche doit etre teste. Toute merge request doit couvrir a 80% le code nouveau ou modifie, et ne pas faire baisser la couverture globale. C'est mesure automatiquement dans la MR Gitlab. Cette regle seule fait monter la couverture mecaniquement.

2. Tests cibles sur les zones a risque. Je croise les 20 fichiers les plus modifies (`git log`) avec le rapport de couverture. Les fichiers tres modifies et peu testes, c'est la que les regressions arrivent. Les seniors testent les modules complexes, les juniors les modules plus simples -- bon moyen d'apprendre le codebase.

3. Chaque bug corrige = un test ecrit AVANT la correction. Zero exception. Ca empeche le bug de revenir et ca construit la couverture sur des cas reels.

Focus specifique au projet :

- Requetes SQL complexes : tests d'integration avec fixtures pseudonymisees
- Interoperabilite FHIR : tests de conformite sur les ressources echangees
- Export de donnees : tests sur les formats de sortie (CSV, FHIR bundles)

Stabiliser les deploiements

Les causes probables des regressions et rollbacks :

Cause	Correction
Merge sans validation automatique	Gate de tests obligatoire dans Gitlab CI
Pas d'environnement de staging	Namespace K8S dedie, deploy auto sur merge dans `develop`
Deploiements semi-manuels	ArgoCD full GitOps
Pas de smoke tests post-deploy	Suite minimale automatisee apres chaque deploy
Pas de deploiement progressif	Canary 10% du trafic via Argo Rollouts, rollout si OK

Par ordre de priorite

Sprint 1-2 (fondations) :

- Gate de tests dans la CI (2h de config, impact immediat)
- Coverage report automatique dans les MR (1h de config)
- Environnement de staging sur K8S (1-2j, on a deja K8S et ArgoCD)

Sprint 3-4 (fiabilisation) :

- Deploiement canary via Argo Rollouts (2-3j)
- Smoke tests post-deploy automatises (2j)
- Feature flags sur les features a risque -- simple table de config, pas d'outil complexe (1j)

Sprint 5+ (optimisation) :

- Tests de performance automatises (requetes critiques avec seuils)
- Environnements de review ephemeres par MR (ArgoCD ApplicationSets)

Ce que ca donne concretement

Mois 1 : Plus de merge sans tests. Staging operationnel. Pipeline complet. Couverture ~55%.

Mois 3 : Canary en place. Smoke tests actifs. Couverture 65%. Rollbacks rares. L'equipe deploie 2-3 fois par semaine sans stress.

Mois 6 : Couverture 75%. Le deploiement est un non-evenement. La dette technique baisse regulierement.

L'objectif n'est pas la perfection technique. C'est que l'equipe retrouve confiance dans ce qu'elle livre.

Comment gerez-vous les problemes dans l'equipe ?

4 personnes sur 9 en difficulte, c'est presque la moitie. Ce n'est pas un probleme RH isole -- c'est un signal systemique. Dans un projet en retard avec de la dette et des deploiements chaotiques, la demotivation est souvent une consequence, pas une cause.

Mon approche : comprendre avant d'agir, traiter chaque situation individuellement, renforcer le cadre collectif.

Semaine 1 : Entretiens individuels

Chaque conversation est confidentielle, sans jugement, orientee solution. Pour chaque personne en difficulte, je cherche la cause racine : est-ce le projet ? Le management precedent ? Un manque de perspective ? Un gap de competence ? Un sentiment de ne pas etre ecoute ?

Traitement par situation

Le dev senior demotive

Ca me fait penser a une situation deja vecue : un senior qui avait vu trop de mauvaises decisions s'empiler sans pouvoir les corriger. Dans ce cas, je l'ecoute, je lui confie un chantier a impact visible avec de l'autonomie (ex: optimisation des requetes SQL), et je le positionne en referent technique. Suivi informel regulier, pas du micro-management.

Le dev junior demotive

Ca me rappelle des situations ou le junior etait isole, pas assez encadre. Si c'est le cas, je mets en place un binomage avec un senior bienveillant, des objectifs clairs et atteignables a chaque sprint, et je valorise ses contributions en sprint review.

Le dev confirme qui sous-performe

Par experience, ca peut venir de plusieurs causes. Entretien franc et bienveillant : "Voici ce que j'observe. Comment tu vois les choses ?" Selon ce qui ressort : plan de montee en competences + binomage, ou revoir le decoupage des taches. Objectifs mesurables et point hebdo pendant 6-8 semaines. Si apres 2 mois il n'y a pas d'amelioration malgre l'accompagnement, repositionnement en coordination avec le management et les RH.

Le dev senior qui refuse l'architecture et cree un climat deletere

J'ai deja rencontre ce profil. C'est le cas le plus sensible, et il est possible qu'il ait raison sur le fond.

Sur le fond : je lui donne un espace structure pour exprimer ses desaccords via les Architecture Decision Records (cf. Q4). S'il a raison, on integre. S'il a tort, on argumente factuellement.

Sur la forme : c'est non-negociable. Entretien direct et respectueux : "Je veux entendre tes desaccords techniques. Mais la facon dont ca s'exprime impacte toute l'equipe. J'ai besoin que tu challenges les idees, pas les personnes."

Si le comportement ne change pas apres 3-4 semaines, escalade au management. Un seul individu ne doit pas mettre en peril 8 collegues et le projet.

Renforcer le cadre collectif

- **Retrospectives reelles** : formats qui liberent la parole (Start/Stop/Continue, Speedboat), pas de "ca va" quand ca ne va pas
- **Responsabilisation** : chaque dev a un domaine technique clair, les seniors mentorent les juniors (formalise, pas optionnel)
- **Reconnaissance** : valoriser les contributions en sprint review, partager les metriques positives
- **Externes (4 sur 9)** : traites comme les internes, pas cantonnes aux taches ingrates. Mais chaque module critique a au moins 1 interne qui le connait

---

Planning

Semaine	Action
S1	Entretiens individuels, diagnostic
S2	Plan d'action par personne, premiers binomages
S3	Cadre collectif (retros, responsabilisation)
S4-S6	Suivi rapproche, ajustements
S6-S8	Bilan : ca marche ou escalade

L'equipe est le principal actif du projet. Investir dans les personnes n'est pas du temps perdu -- c'est la condition pour que tout le reste fonctionne.

Certains developpeurs veulent reecrire toute la stack back-end, il y a des desaccords d'architecture, comment reagissez-vous ?

Ma reponse courte : non, on ne reecrit pas. Mais on ne balaye pas les arguments sous le tapis non plus.

Pourquoi la reecriture est un piege ici

Facteur	Reecriture complete	Refactoring progressif
Risque	Tres eleve	Maitrise
Delai	+6-12 mois minimum	Continu
Valeur metier pendant le chantier	Zero	Continue a chaque sprint
Budget	Hors cadre	Integre (20% dette/sprint)
Regression	Tres probable	Controlee par les tests

La reecriture complete est presque toujours une erreur strategique quand le projet est en retard, le budget contraint, la plateforme en production, et l'equipe en tension.

Ce que je dis a l'equipe : "Je comprends la frustration. Mais reecrire le back-end, c'est 6 mois minimum ou on ne livre rien, avec un projet deja en retard. En revanche, on va ameliorer systematiquement ce qu'on touche, et prendre les bonnes decisions d'architecture pour la suite."

Mon approche : migration progressive (Strangler Fig Pattern)

Au lieu de reecrire, on enveloppe et remplace progressivement :

1. Les **nouveaux modules** sont ecrits en FastAPI si c'est justifie techniquement
2. Les **modules existants** sont migres vers FastAPI **quand on les touche** pour une feature ou un fix
3. L'ingress K8S route le trafic vers Django ou FastAPI selon l'endpoint
4. Aucun big bang. La migration se fait sur 12-18 mois, transparente pour les utilisateurs

Criteres de choix Django vs FastAPI :

- CRUD simple avec admin → Django (on garde)
- API haute performance, requetes complexes → FastAPI (async natif)
- Module avec beaucoup de logique existante → Django (risque de regression)
- Interop FHIR (I/O bound) → FastAPI

Structurer les debats : Architecture Decision Records (ADR)

Les desaccords se reglent aujourd'hui par la force de conviction. Je mets en place les ADR : un document court (1 page) par decision d'architecture, avec contexte, options, avantages/inconvenients, et decision.

Regles du jeu :

- Tout le monde peut proposer un ADR, senior ou junior
- Les arguments sont factuels : benchmarks, contraintes projet (pas "je prefere X")
- Delai de decision : 1 semaine max
- Si pas de consensus, le Delivery Manager tranche et assume

- Une decision prise est respectee par tous. On peut la revisiter avec un nouvel ADR, pas en la sabotant

Pour le dev senior contestataire, ce cadre lui donne un espace legitime d'expression et des criteres objectifs, mais aussi des limites claires.

Plan d'action

Semaine 3-4 : Lancer le premier ADR "Strategie de coexistence Django/FastAPI". Le dev senior contestataire est invite a co-rediger. Identifier les 3-5 modules candidats a la migration.

Mois 2-3 : Migrer un premier module pilote vers FastAPI (petit, bien delimite, a forte valeur). Le dev senior contestataire peut le porter. Mesurer performance, temps de dev, couverture de tests avant/apres.

Mois 4+ : Decider de la suite sur la base des resultats reels. Si FastAPI apporte les gains attendus, on continue la migration progressive. Sinon, on ajuste.

Le vrai enjeu n'est pas "Django vs FastAPI". C'est d'avoir un cadre ou les desaccords techniques se transforment en decisions eclairees, pas en conflits.

La deadline ne peut pas bouger. Le scope non plus. Que faites-vous ?

Deadline fixe. Scope fixe. Projet en retard. Sacrifier la qualite sur une plateforme de donnees patients, c'est non.

Ma conviction : quand on dit "le scope ne bouge pas", ca signifie que les objectifs metier ne bougent pas. La facon d'atteindre ces objectifs est toujours negociable.

---

5 leviers

1. Negocier le "comment", pas le "quoi"

Je m'assois avec le PO et les utilisateurs cles pour definir le MVP de chaque feature :

Feature	Version complete	Version MVP (meme valeur metier)
Requetage	15 types de filtres	5 filtres les plus utilises + requete SQL pour le reste
Visualisation	8 graphiques interactifs	3 graphiques essentiels + export pour analyse externe
Export	6 formats	3 formats (CSV, FHIR, Excel) -- couvrent 90% des usages

On livre 100% de la valeur metier avec 60-70% de l'effort.

2. Paralleliser les flux de travail

Je reorganise l'equipe en flux autonomes :

- **Flux A** (2 seniors) : Requetage + Performance
- **Flux B** (1 senior + 2 confirmes) : Visualisation + Export
- **Flux C** (1 confirme + 1 senior) : Infra + Qualite + FHIR
- **Flux D** (2 juniors + support senior) : Frontend

Chaque flux a un lead, un objectif clair, et avance en parallele.

3. Renforts cibles

Pas de recrutement massif (loi de Brooks), mais 2 renforts temporaires sur des perimetres isoles :

- 1 dev senior FastAPI (freelance, 3-4 mois) → debloque le moteur de requetage
- 1 DevOps/SRE (mi-temps, 2-3 mois) → stabilise CI/CD et monitoring

Cout : ~15-20% du budget restant. C'est un investissement.

4. Eliminer le gaspillage

Gaspillage	Action
Reunions inutiles	Audit, suppression du superflu
Specs floues → allers-retours	Definition of Ready stricte
Code reviews qui trainent	Review dans la demi-journee
Context-switching	WIP limit : max 2 taches/dev
Attente de validation metier	PO disponible chaque jour

Estimation conservatrice : 15-20% de capacite effective recuperee.

5. Livrer incrementalement

3 releases, chacune deployable et utilisable :

- **Release 1 (M1-2)** : Requetage de base, visu simple, export CSV, stabilisation CI/CD
- **Release 2 (M3-4)** : Requetage avance, 3 types de visu, export FHIR + Excel, interop FHIR
- **Release 3 (M5-6)** : Dashboards perso, requetes predefinies, scale x3

Si la deadline arrive et que R3 n'est pas 100% terminee, on a quand meme une plateforme fonctionnelle en production.

La conversation avec le DSN

Je ne vais pas le voir en disant "on est en retard". J'y vais avec un plan : "Le scope fonctionnel sera livre. Voici comment. Le risque residuel est sur les features les moins prioritaires de R3 -- si on doit arbitrer, voici lesquelles. Je propose qu'on valide ensemble."

Le DSN veut de la visibilite et du controle, pas des mauvaises surprises.

La seule chose que je refuse de negocier : la qualite et la securite des donnees patients. C'est une ligne rouge absolue.

Comment replanifiez-vous le reste a faire, les activites des equipes, le budget ?

Point de depart

Indicateur	Valeur
Temps ecoule	3 mois (~25%)
Budget consomme	25%
Fonctionnalites livrees	15%
Temps restant	~9 mois
Budget restant	75%

Constat : au rythme actuel (5%/mois), on livrerait 60% du scope en 12 mois. Il faut doubler la vitesse effective.

---

1. Reste a faire

Inventaire : lister toutes les features non livrees, estimer en T-shirt sizing (S/M/L/XL), prioriser. Evaluer l'avancement reel des features en cours. Categoriser les 120 tickets de dette (bloquant / structurant / cosmetique).

Capacite realiste : 9 devs, sprints de 2 semaines, ~7 jours productifs par dev par sprint. Avec 20% dette + 10% buffer = ~44 j/h de feature par sprint. Sur 9 mois (18 sprints) : ~800 j/h. Avec renforts (+2 ETP, 3 mois) : ~920 j/h.

Decoupage en 3 releases :

- **Release 1 (M4-M5)** : Requetage de base, visu simple, export CSV, CI/CD stable (~35% scope, ~300 j/h)
- **Release 2 (M6-M7)** : Requetage avance, 3 visu, export FHIR/Excel, interop FHIR (~30% scope, ~350 j/h)
- **Release 3 (M8-M9)** : Dashboards perso, requetes predefinies, scale x3 (~20% scope, ~270 j/h)

---

2. Organisation des equipes

4 flux de travail dedies (cf. Q5) :

Flux	Equipe	Perimetre
A	2 seniors (+ renfort FastAPI)	Requetage + Performance
B	1 senior + 2 confirmes	Visualisation + Export
C	1 confirme + 1 senior (+ renfort DevOps)	Infra + Qualite + FHIR
D	2 juniors (support senior)	Frontend

Roles cles : je pilote le delivery, le PO priorise et valide, 2 tech leads (back + front) arbitrent les choix techniques. Les externes sont repartis comme les internes, mais chaque module critique a au moins 1 interne qui le connait.

---

3. Budget

Situation actuelle (hypothese) :

Poste	Initial	Consomme	Reste

Equipe interne	50%	12.5%	37.5%
Prestataires externes	35%	10%	25%
Infra + licences	10%	2%	8%
Marge	5%	0.5%	4.5%

Reallocation : renforts temporaires (8%), outillage monitoring (2%), formation (1%). Ca depasse de ~~8%. Pistes d'economies : fin de contrat d'un presta si non-performance averee (~~6%), optimisation infra K8S (~2%), reduction de la duree des renforts si le plan avance bien.

Je presente au DSN 2 scenarios :

- **Scenario A** (recommande) : renforts + optimisations = budget a 100%, scope complet
- **Scenario B** (degrade) : sans renforts = budget a 95%, scope reduit de 10-15%

---

4. Ceremonies de pilotage

Ceremonie	Frequence	Objectif
Backlog refinement	Hebdo, 1h	Estimer, decouper
Sprint planning	Bi-hebdo, 2h	Engagement equipe
Release planning	Toutes les 6 semaines	Valider le contenu de la release
Budget review	Mensuel	Suivi consomme vs planifie
Comite de pilotage	Mensuel	Arbitrages avec le DSN

Le plan sera faux dans le detail -- tous les plans le sont. Mais le cadre de suivi permet de s'adapter sprint apres sprint.

Comment structurez-vous le reporting ?

Le reporting repond a 3 questions : ou en est-on ? Quels sont les risques ? De quelles decisions a-t-on besoin ?

Principe : chaque audience a un reporting adapte a ses decisions.

Niveau 1 : Equipe (quotidien / sprint)

Daily standup (15 min) : synchronisation, pas reporting. Chaque dev dit ce qu'il a fait, ce qu'il fait, ce qui le bloque.

Metriques de sprint (automatisees via Gitlab/Jira) : burndown, velocite sur les 6 derniers sprints, WIP par dev (cible < 2), cycle time, taux de completion. Ces metriques sont pour l'equipe, pas pour le management.

Niveau 2 : Comite delivery (hebdo, 30 min, DM + PO + Tech Leads)

Agenda fixe :

1. **Dashboard projet (5 min)** -- statut scope/delai/budget en vert/orange/rouge avec tendance, metriques cles du sprint (points completes, couverture tests, temps de requete, deploys et rollbacks)
2. **Risques et blocages (10 min)** -- chaque risque a un owner, une action, une echeance. Pas de risque "a surveiller" sans plan
3. **Decisions a prendre (10 min)** -- arbitrages presentes avec contexte, options, et ma recommandation. Decision prise seance tenante si possible
4. **Actions (5 min)** -- recap, owners, echeances. Suivi d'une semaine sur l'autre

---

Niveau 3 : Comite de pilotage (mensuel, 45 min, DM + DSN + PO)

Rapport executif (1 page, envoye 24h avant) : statut global, avancement scope/budget/delai, faits marquants du mois, risques majeurs, decisions requises, previsions mois suivant.

Agenda : revue du rapport (10 min), demo si release livree (10 min), risques et arbitrages (15 min), projection (10 min).

Regle : le DSN ne decouvre jamais un probleme en comite de pilotage. Si c'est urgent, je l'appelle avant.

Metriques suivies

DORA :

Metrique	Cible M3	Cible M6
Frequence de deploiement	2-3/sem	3-5/sem
Lead time (commit → prod)	< 3 jours	< 1 jour
Taux d'echec des deploys	< 10%	< 5%
MTTR	< 2h	< 30 min

Specifiques projet : couverture de tests (65% M3, 75% M6), temps de requete P95 (< 3s M3, < 2s M6), tickets de dette (< 80 M3, < 50 M6), bus factor (>= 2 par module).

Outillage

Gitlab Issues/Jira pour le board, Gitlab CI pour les metriques de couverture et DORA, Grafana + Prometheus pour le monitoring applicatif. Rapport executif en template Markdown versionne. Pas d'achat d'outil supplementaire.

Le meilleur reporting est celui qui rend les bonnes decisions evidentes.

Proposez une roadmap technique sur 6 mois au format Gantt

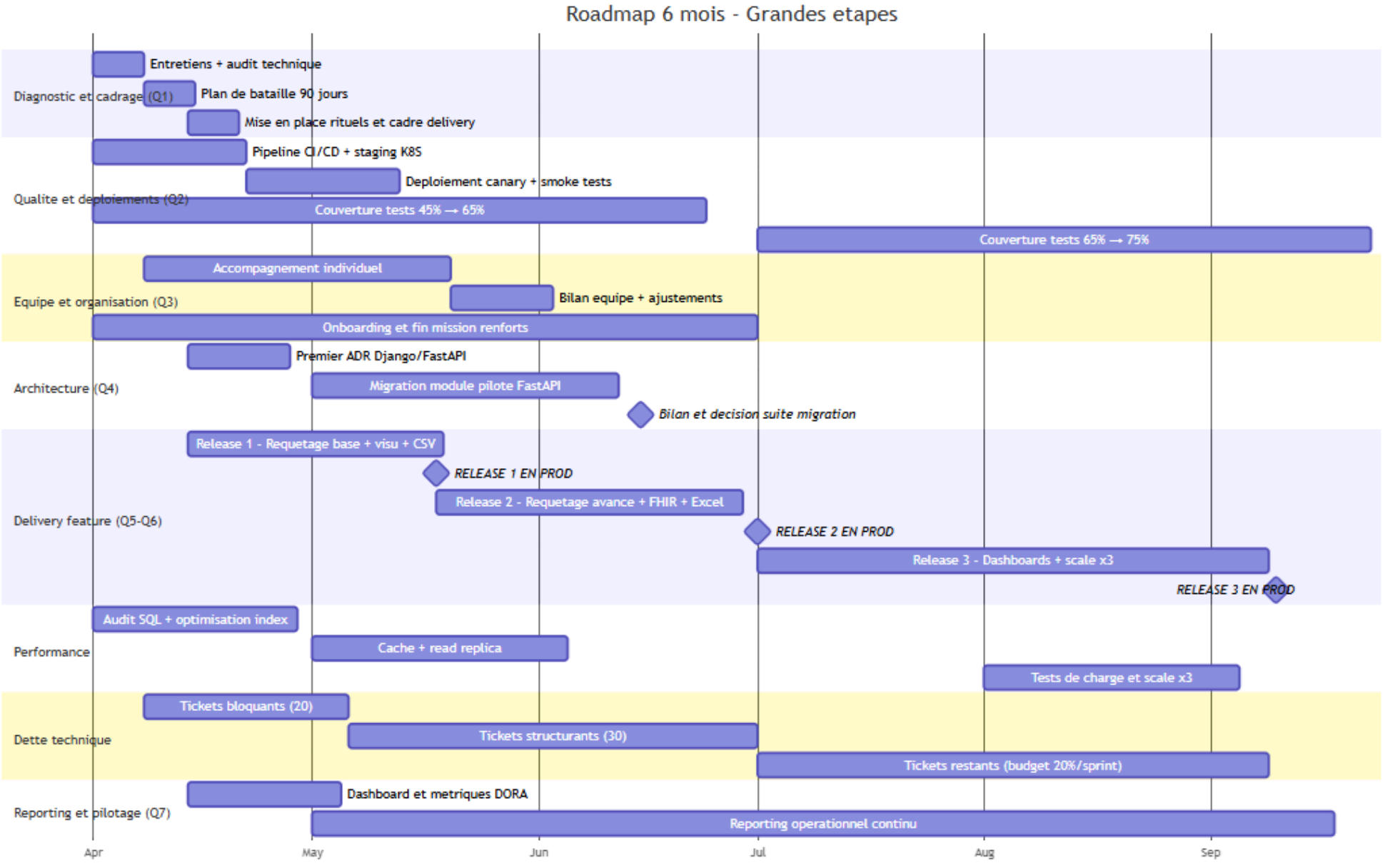
Hypotheses

- Mois 1 = Mois 4 du projet (on reprend le controle apres 3 mois passes)
- Sprints de 2 semaines, equipe de 9 devs + 2 renforts temporaires (mois 1-3)
- Allocation par sprint : 70% features / 20% dette technique / 10% buffer

---

Gantt - Vue grandes etapes

Les sections reprennent les chantiers evoques dans les reponses precedentes : diagnostic (Q1), qualite et deploiements (Q2), equipe (Q3), architecture (Q4), delivery feature (Q5-Q6), reporting et pilotage (Q7).


Roadmap 6 mois - Grandes etapes
Diagnostic et cadrage (Q1)
Entretiens + audit technique
Plan de bataille 90 jours
Mise en place rituels et cadre delivery
Qualite et deploiements (Q2)
Pipeline CI/CD + staging K8S
Deploiement canary + smoke tests
Couverture tests 45% → 65%
Couverture tests 65% → 75%
Equipe et organisation (Q3)
Accompagnement individuel
Bilan equipe + ajustements
Onboarding et fin mission renforts
Architecture (Q4)
Premier ADR Django/FastAPI
Migration module pilote FastAPI
Bilan et decision suite migration
Delivery feature (Q5-Q6)
Release 1 - Requetage base + visu + CSV
RELEASE 1 EN PROD
Release 2 - Requetage avance + FHIR + Excel
RELEASE 2 EN PROD
Release 3 - Dashboards + scale x3
RELEASE 3 EN PROD
Performance
Audit SQL + optimisation index
Cache + read replica
Tests de charge et scale x3
Dette technique
Tickets bloquants (20)
Tickets structurants (30)
Tickets restants (budget 20%/sprint)
Reporting et pilotage (Q7)
Dashboard et metriques DORA
Reporting operationnel continu
Apr
May
Jun
Jul
Aug
Sep

Jalons cles

Jalon	Date cible	Ce qu'on valide
Cadre delivery en place	Mi-avril	Rituels, CI/CD, staging, plan 90j
Release 1 en prod	Mi-mai	Requetage de base, visu, export CSV
Bilan equipe	Fin mai	Accompagnement individuel : ca marche ou escalade
Decision architecture FastAPI	Mi-juin	Module pilote mesure, decision suite migration
Couverture 65%	Fin juin	Mesure automatisee
Release 2 en prod	Debut juillet	Requetage complet, FHIR, exports
Fin mission renforts	Fin juillet	Transfert de competences fait
Release 3 en prod	Mi-septembre	Dashboards, scale x3, plateforme complete
Couverture 75%	Fin septembre	Mesure automatisee

Points de controle (Go / No-Go)

Date	Decision	Critere
Fin avril	Go Release 1	CI/CD stable, perf ameliorees, equipe cadree
Mi-mai	Go Release 2	Release 1 en prod sans incident
Debut juillet	Go Release 3	Release 2 en prod, FHIR OK
Debut aout	Go scale x3	Tests de charge OK, monitoring OK

Chaque Go/No-Go est une decision partagee avec le DSN.

Cette roadmap est une boussole, pas un plan fige. Le detail s'affine sprint apres sprint, mais les grands jalons et les chantiers sont poses.